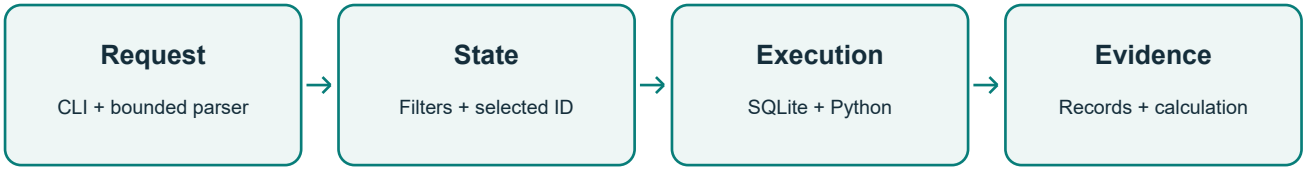
RECIPE BOX

A grounded recipe assistant

Completed implementation, detailed roadmap, framework inventory and reproducible evaluation



Hard restrictions define eligibility before any ranking or display limit.

125,164	122,935	17 + 17
raw source records	searchable records	unit + full-data checks

Prepared 7 November 2026

Python + SQLite FTS5. No runtime LLM API or API key. All three official Recipe Box files were imported.

Published: github.com/quackapuss/recipebox-grounded-chatbot. Public cloning and GitHub Actions verified the uploaded source.

Contents

1. What was built 3

2. The work sequence and its purpose 3

3. Every framework, algorithm, and tool used 12

4. Database schema walkthrough 14

5. Request walkthroughs 14

6. How to reproduce every important result 15

7. Limits, tradeoffs, and extension roadmap 16

8. Interview walkthrough and exercises 16

Dataset coverage 17

Exact analytical results 17

Executed conversations and timing 18

Evidence-backed adaptation 18

What initially failed and why 19

Final checks 20

One-shot answer comparison 20

What worked and what remains limited 21

Reproduce 21

Deliberate scope limits 23

AI assistance disclosure 24

Submission status 24

Detailed implementation roadmap and explanation

1. What was built

This project implements a local conversational recipe assistant for the supplied GenAI internship assignment. It searches existing Recipe Box records, answers supported factual requests, computes exact dataset statistics, matches a pantry against existing recipes, and proposes evidence-backed complete recipe variants when a selected ingredient must be removed. Conversation state keeps relevant filters and restrictions across turns.

The central design rule is that every dataset claim must have a data path. An original ingredient or instruction is copied from a source record. A statistic is calculated over an explicitly defined population. A match is derived from explicit set operations. A proposed adaptation cites the local recipe that supplies its complete ingredient list and preparation method. If the system cannot establish the necessary evidence, it reports the gap.

The delivered implementation is deliberately reproducible without an API key. It uses Python's standard library and SQLite's FTS5 extension. An LLM API, LangChain, a vector database, and a web framework are not runtime requirements. The assignment permits architectural choice; this implementation uses that freedom to make grounding and computation easy to inspect. The natural-language grammar is bounded, and this is a documented limitation rather than a claim of universal language understanding.

Read the root README first to run the app. Read `REQUIREMENTS.md` to see how each brief requirement maps to code. Read `EVALUATION.md` and `test_conversations.md` for measured results and actual responses. This roadmap explains what was done, why, the complete framework inventory, the algorithms, and how to reproduce or extend each part.

2. The work sequence and its purpose

Step 1: Obtain and read the actual brief

The supplied Google Drive link was first opened through a web reader. That reader could not access the file. A direct Drive download succeeded and produced the four-page assignment PDF. All four pages were extracted and rendered so that requirements, examples, evaluation conditions, AI disclosure, and submission instructions were considered together.

Why: implementation choices should respond to the actual assignment. Starting from a guessed problem statement could miss the all-three-files rule, complete-population analytics, evidence requirements for substitutions, or the public-repository requirement.

Output: a requirement mapping covering the three weighted tasks, shared data and conversational requirements, evaluation, disclosure, and submission. The local source brief was used for understanding; it is not republished with the project.

Step 2: Inspect the official dataset source

The official Recipe Box page and the publisher's repository were checked. The official archive was downloaded from `https://eightportions.com/recipes_raw.zip`. Its members are three JSON objects and a license. The files correspond to Allrecipes, Epicurious, and Food Network.

Why: all three sources are explicitly required. A convenient small sample would not establish full-dataset counts or ranking. Using the publisher's archive also avoids introducing a third-party repackaging with a different schema or unknown record coverage.

Output: a local copy of all three source files, their SHA-256 checksums, and a data manifest. No original source URLs are invented: the distributed dataset uses hashed source keys.

Step 3: Inspect schema and data quality

Representative records were inspected before designing the database. Recipes normally contain `title`, `ingredients`, `instructions`, and `picture_link`. Some records are null or incomplete. Some Allrecipes ingredient strings contain `ADVERTISEMENT`, including standalone advertisement entries. Instructions are strings whose paragraph conventions vary by source. Standardized nutrition, serving yield, and cooking-time columns are absent.

Why: these details affect every downstream answer. Counting advertisement lines as ingredients inflates statistics. Treating every record as valid causes errors. Assuming newline-only instruction steps creates inconsistent rankings. Inventing missing nutrition would contradict the brief.

Output: explicit validation, normalization, instruction-step, unavailable-attribute, and denominator policies. A recipe is searchable only if it has a nonempty string title, at least one usable ingredient line, and nonempty string instructions. Invalid records remain stored with their stable ID and original raw value.

Step 4: Choose an inspectable architecture

The application is divided into ingestion/storage, normalization, conversation routing/state, matching, adaptation, rendering, CLI, and evaluation modules. SQLite stores the complete records and indexes. FTS5 handles lexical discovery. SQL and Python calculate results. A dataclass stores the current conversation state.

Why: grounding requires the retrieval and calculation path to remain separate from language interpretation. A fluent model response is not a reliable aggregate engine. Modular code also makes interview modifications practical: a new alias belongs in normalization, a new statistic belongs in the store, and a new language expression belongs in the parser.

Output: a dependency-free Python package runnable with `python -m recipebox`, plus tests and explanatory documents. Python 3.11 or newer is required.

Step 5: Define stable identity and provenance

Every source key is namespaced with its source: `ar:original-key`, `epi:original-key`, or `fn:original-key`. The database stores the source, source key, original fields, and serialized original record. Derived ingredient names and step counts are separate columns and tables.

Why: a title can change, repeat, or differ only in punctuation. It is unsuitable as an ID. A namespace prevents a source-key collision across files. Keeping originals separate allows any normalized result to be audited and prevents cleaning rules from erasing evidence.

Output: stable citation targets, lossless source-value preservation, and a predictable `/show ID` operation. JSON formatting and key order may change during serialization, but the original JSON values are retained; this is not a byte-for-byte copy of the source file.

Step 6: Implement reproducible ingestion

`setup` downloads the archive if any required JSON file is missing. It extracts only the expected filenames and the license. `build` validates that all three files exist before starting. It writes a temporary `.building` database, loads one source JSON object at a time, commits source batches, and replaces the active database only after successful completion.

Why: selecting only known archive members avoids arbitrary archive extraction paths. Requiring every source prevents accidental partial populations. A temporary database prevents a failed rebuild from replacing an existing working database. Reading one source at a time keeps memory below loading all three simultaneously, though this is not a streaming JSON parser.

Output: a complete SQLite database and manifest containing per-source raw, valid and invalid counts, checksums, normalization version, duplicate-content observations, and invalid example IDs. Source-content duplicates are retained so statistics count source records, not inferred unique dishes.

Step 7: Normalize ingredient text conservatively

Normalization lowercases and normalizes Unicode, expands common fraction characters, removes advertisement text, strips a leading quantity and measurement unit, removes parenthetical/comma preparation notes, removes a bounded set of preparation adjectives, and applies a small documented alias/singularization vocabulary. Examples include onions to onion and scallion to green onion.

Why: strings such as `2 onions, finely chopped` and `1 onion` should often match at the name level. However, garlic powder and garlic, egg whites and eggs, cooked rice and rice, and tomato paste and tomato must remain distinct products. Broad stemming of ingredient identity would incorrectly merge these cases.

Output: exact normalized product names attached to the original ingredient line and its position. Unknown or complicated product phrases remain relatively specific. This sacrifices some recall to avoid claiming unjustified equivalence. The parser is heuristic, not a comprehensive ingredient ontology.

Step 8: Distinguish identity from family filtering

Pantry matching uses exact normalized product identity. Ingredient include filters use whole-token phrase membership. Therefore `garlic` as a filter also matches `garlic powder`, while pantry garlic does not automatically satisfy a recipe's garlic-powder requirement.

Why: analytical questions often ask about ingredient families, while physical pantry matching requires product distinctions. Combining these semantics would either miss relevant analytical variants or overstate pantry coverage.

Output: two explicit rules. For example, a tomato-family statistic may include tomato paste, but tomato paste remains a separate product in a pantry result. The chatbot prints the analytical rule so the user can reproduce and interpret the number.

Step 9: Define ingredient and instruction counts

An ingredient count is the number of usable normalized ingredient lines. Duplicate lines count separately for this measurement. A common-ingredient frequency instead counts distinct recipe IDs per exact normalized product, so repeated garlic lines in a recipe do not give garlic multiple votes.

Instruction counting uses nonempty newline paragraphs when there is more than one. For a single paragraph, it splits at `.`, `?`, or `!` followed by whitespace and an uppercase letter or digit. A returned step count is a text-segmentation measure, not a validated estimate of cooking difficulty.

Why: the dataset does not have a uniform numbered-step field. A deterministic documented convention is more defensible than allowing a model to decide how many steps a recipe appears to contain. The distinction between ingredient-line count and distinct-product frequency prevents ambiguous analytics.

Output: precomputed counts that can be sorted and averaged, with the exact rule attached to analytical responses. Abbreviations, unusual punctuation, headings, and source paragraphs can affect counts; those limits are disclosed.

Step 10: Build lexical search

FTS5 indexes recipe title and normalized ingredient text with the Porter/Unicode tokenizer. A user dish query is converted into quoted lexical terms combined with AND. Search ranking uses FTS5 BM25, with title weight 5 and ingredient weight 1; ties use stable ID. The title is weighted more strongly because dish names should influence discovery more than incidental ingredients.

Why: this is a small, local, auditable retrieval mechanism. It supports dish names and ingredient words without an embedding model download. Quoting extracted tokens prevents user input from becoming arbitrary FTS syntax. Query values are bound through SQL parameters.

Output: existing recipe IDs, titles, ingredient evidence, and counted steps. Full ingredients and instructions are available through `/show`. This retrieval is lexical: an unfamiliar paraphrase or conceptual request may fail. The program does not silently broaden AND terms into OR terms.

Step 11: Implement complete-population filtering

`Store.predicate` is the shared filter builder. It combines searchable-record status, optional full-text query, every included ingredient family, every excluded ingredient/group, and an optional maximum step count. The same filter semantics are used for search, statistics, and pantry matching.

Why: if search and statistics interpreted restrictions differently, a user could receive a count that contradicted the visible eligible recipes. Reusing the population definition makes that relationship explicit. A hard restriction must be applied before ranking or top-k truncation.

Output: parameterized SQL predicates and arguments. The column names and ordering choices come from fixed code, not a language model. Exclusion groups for dairy, egg, and peanut are precomputed during ingestion; custom exclusions use normalized whole-token product matching.

Step 12: Implement exact analytics

The analytical method obtains every filtered recipe, computes `n`, and reports the denominator as all searchable records across all three sources. Count and percentage are `n` and `100 * n / denominator`. Mean is the sum of ingredient-line counts divided by `n`. Median sorts those counts and takes the middle value or the mean of the two middle values. Common ingredients use SQL `COUNT(DISTINCT recipe_id)` grouped by normalized name.

Step rankings order every filtered record by step count and stable ID, then display the requested number. The display limit never defines the aggregate population. Empty mean and median return null rather than zero. A zero denominator produces no percentage rather than division by zero.

Why: a semantic retriever's nearest five results are a sample selected by relevance, not a valid basis for a dataset statistic. Complete-population computation is a central requirement of the brief. Each response includes filters, recipes analyzed, calculation, and example evidence IDs; the filter and manifest, rather than a few examples, define the aggregate's full provenance.

Output: reproducible analytical JSON. Actual measured garlic/onion and tomato results are recorded in `EVALUATION.md` rather than guessed in this roadmap.

Step 13: Implement explicit conversation state

`State` stores query, included ingredient families, excluded ingredients/groups, maximum steps, pantry, ranking preference, current mode, last displayed result IDs, and selected recipe ID. The first displayed result becomes selected. `/show 2` or a stable ID changes the selection.

Why: a follow-up such as `I cannot have dairy` has no standalone dish name. Its meaning depends on the prior search. State is structured rather than reconstructed from a growing prompt, so the user and tests can inspect exactly which restrictions are active.

Output: follow-ups rerun the active search or pantry operation with updated constraints. `/scope` shows current state. New dish or pantry searches replace dish/pantry context but preserve restrictions and maximum steps. `/allow ingredient`, `/maxsteps off`, and `/reset` are explicit ways to remove restrictions. Session state lasts for that CLI process, not across restarts.

Step 14: Implement bounded natural-language routing

The router recognizes patterns for searches, ingredient availability, exclusions, maximum steps, preference words, count/percentage, mean/median/common ingredients, step rankings, recipe selection, and ingredient removal/replacement. Ordinal and selected-recipe references use state. The example conversations in the brief are covered.

Why: deterministic routing removes API cost, network dependence, and model-generated query errors. It makes the assignment's core grounding behavior testable without credentials. The tradeoff is language coverage. The system does not pretend to understand every culinary request.

Output: a validated plan with action and typed fields. Exact slash commands and `/json` offer reproducible control when wording falls outside the grammar. Unknown input leaves the state unchanged and points to help. Invalid plans are rejected before state mutation; later supported user-input failures roll state back.

Step 15: Implement ingredient-set matching

Let A be the set of exact normalized pantry products and R be the set of exact normalized required products for a recipe. Matched products are `A intersection R`. Missing products are `R minus A`. Unused pantry products are `A minus R`. Extra required ingredients are the same products as missing: these are what the user would need to obtain. The response names both terms so there is no hidden extra pantry allowance.

Every eligible existing recipe sharing at least one pantry product is considered. The engine does not invent a recipe and does not assume salt, oil, or water are automatically available. Hard constraints and maximum steps apply before ranking.

Why: set differences make pantry coverage explainable. Requiring a nonzero overlap avoids suggesting arbitrary recipes when nothing matches. Keeping staples explicit is more honest than silently treating them as free ingredients.

Output: stable recipe IDs, matched/missing/unused products, coverage, counted steps, original evidence, and a rank key for every displayed result. Required quantities are handled separately from identity.

Step 16: Make ranking preferences visible

Default ranking is lexicographic: fewest missing products, most matched products, greatest coverage, fewest steps, then stable ID. `use_most` prioritizes most pantry products used before missing count. `fewest_steps` prioritizes counted steps before missing count. Coverage is `100 * matched / distinct required products`.

Why: a hidden weighted sum can be difficult to defend. With lexicographic ordering, a preference's meaning is visible and tie-breaking is deterministic. Missing count and coverage answer different questions: a high percentage does not necessarily mean the fewest products need to be bought.

Output: the ranking policy and numeric key are displayed. No key measures flavor or objective culinary quality. The ranking is a transparent utility policy for the stated request, not a learned preference model.

Step 17: Compare quantities only where justified

The quantity parser recognizes simple integers, decimals, fractions, and mixed fractions followed by recognized measurement units. It converts compatible masses to grams and compatible volumes to milliliters using explicitly US customary volume factors. Simple egg/onion/tomato/banana counts are supported. Duplicate comparable lines are summed for a product.

Ranges, package expressions such as `1 (14 ounce) can`, missing values, and incompatible dimensions are not verified. No ingredient-density conversion is attempted. A known comparable shortage is reported as insufficient. Name-level ranking remains separate from quantity checks; an insufficient quantity does not silently become sufficient.

Why: comparing 500 grams of rice with a recipe's cup measurement requires density and preparation context absent from the dataset. Guessing that conversion would give false confidence. Similarly, a package count is not a reliable weight or volume without parsing the exact packaging.

Output: per-product sufficient, insufficient, or not-verified status, with canonical amounts where reliable. These are parse-based checks, not guarantees about measurement variability or available serving yield.

Step 18: Retrieve local substitution evidence first

The adaptation method receives a selected base recipe, target ingredient, and active exclusions. It verifies that the target appears in the base ingredient text. It identifies a bounded role hypothesis for egg, cream, butter, or milk. The role map suggests candidate product families to look for, but is not itself evidence.

The local search uses dish-title anchors: banana bread specifically requires both words; common dish terms such as pasta or bread can anchor other searches. A candidate donor must avoid every exclusion and the removed target, share at least three product families with the base-without-target, and reach a Jaccard similarity of at least 0.45. For this similarity calculation only, bleached/unbleached all-purpose flour share a family. Exact pantry identities and reported ingredient differences remain distinct. At least one method word must occur in both instructions. A newly present role candidate must occur in the donor.

Why: mere co-occurrence of an ingredient somewhere in 125,000 records does not justify its use as a replacement. Related dish identity, ingredient overlap, and preparation evidence provide a more relevant screen. These hand-chosen thresholds are conservative heuristics, not calibrated probabilities.

Output: ranked local donor records with IDs, shared ingredients, Jaccard score, method words, candidate products, additions, removals, and uncertainty. Active exclusions are checked in donor ingredient lists and instruction text so a method mentioning a forbidden ingredient is not silently copied.

Step 19: Return a complete observed variant

When evidence passes, the revised ingredient list and instructions are the complete cited donor recipe. The response displays the base ID and donor ID, explains all product differences, and tells the user to use the donor's own quantities and method. It does not splice a guessed ratio into the base or assume the base's baking method remains valid.

Why: local recipes are observational examples, not controlled substitution experiments. A donor may differ in sugar, fat, liquid, or baking sequence as well as the target ingredient. Copying its complete preparation keeps the output grounded in an observed dataset record while showing those confounders.

Output: one or more complete related-variant adaptations. This meets the revised-list/method requirement conservatively, but it is not unrestricted recipe synthesis. If a reviewer expects isolated ingredient edits that preserve the rest of the base, that remains an explicit limitation and an extension point.

Step 20: Communicate uncertainty and abstain

The system distinguishes original facts, calculated results, heuristic role hypotheses, indirect variant evidence, and predicted taste/texture effects. No precise nutrition is manufactured. Weak local substitution evidence produces an unsupported response instead of a plausible replacement. The runtime does not use external internet search for culinary substitutions.

Why: the assignment values reliable refusal above fabrication. Ingredient text also cannot establish undeclared product ingredients, manufacturing cross-contact, or medical allergy safety. Those missing facts should not be disguised as a positive safety determination.

Output: reasoned gaps and stated limits. A failed adaptation does not quietly relax dairy or egg restrictions. No constraint-violating alternatives are offered automatically; the brief allows omitting alternatives when no eligible result exists.

Step 21: Test using a known-answer miniature dataset

The unit suite constructs three small synthetic source files with known ingredient lists, duplicate source keys, a null record, scraper noise, dairy/plant variants, a pantry conflict, and a structurally egg-dependent dish. It builds a temporary real SQLite database and drives the actual chatbot.

Why: an independent small oracle can assert exact counts and exclusions without copying large-data outputs into a test. More than five eligible garlic/onion records ensure the test can catch accidental top-five aggregation. Tests also exercise source preservation, quantity uncertainty, form distinctions, selected recipe behavior, and atomic validation failures.

Output: executable `unittest` cases that run without the internet or an API key. `EVALUATION.md` records the final test count and results. Selected-recipe ingredient facts and counts are explicitly tested so that a question about one recipe cannot accidentally become a dataset count. The GitHub workflow specifies Python 3.11, 3.12, and 3.13; only locally executed versions are claimed as verified.

Step 22: Evaluate against the full dataset

The evaluation runner executes eight multi-turn conversations spanning all three tasks. It includes the assignment's search and restriction example, exact count/percentage, tomato step ranking plus mean/median/frequency follow-ups, pantry matching plus egg exclusion, no-match plus empty mean, local banana-bread adaptation, weak meringue evidence, and missing nutrition/quantity reporting.

An independent Python population scan checks the garlic/onion count against the SQL-backed result. It reuses the documented normalization predicate, so it independently checks population traversal and aggregation rather than claiming an independent linguistic ontology. The selected banana base is deterministic by title length and ID among egg-containing banana breads; the harness does not pick the donor.

Why: unit tests establish controlled logic; real-data execution reveals messy source conventions and retrieval gaps. Recording actual responses prevents the demonstration from becoming a list of imagined successful conversations.

Output: `evaluation_results.json`, `test_conversations.md`, and a human-readable evaluation report, including failures and qualifications rather than an invented benchmark score.

Step 23: Diagnose and fix issues exposed by real-data evaluation

The initial real-data run passed 15 of 16 checks. Its banana-bread adaptation check failed because the related Food Network variant used unbleached all-purpose flour while the base used all-purpose flour. Exact pantry identity was appropriate, but using that same identity for related-variant similarity reduced the overlap below the evidence threshold. A narrowly scoped flour-family projection was added only to adaptation similarity. The exact original differences remain visible, and pantry matching is unchanged. The same base then retrieved the locally observed soy-yogurt variant. The threshold was not lowered to force a pass.

The common-ingredient query also repeated population filters while joining individual ingredient rows. It was changed to select the eligible recipe IDs once in a subquery before grouping ingredient frequency. In the observed runs, the four-turn tomato-statistics scenario improved from about 60 seconds to about 16 seconds; timings are environment-dependent. The independent aggregate oracle was changed from many per-record SQL calls to a single ordered ingredient scan while retaining Python aggregation.

Why: a known-answer miniature dataset cannot reproduce every source naming convention or full-data performance characteristic. Failures should change a specific mistaken assumption or inefficient execution path, then be rechecked against both fixtures and real records. The final report records the final execution, not an invented claim that the first attempt worked perfectly.

Step 24: Demonstrate the grounding problem in a one-shot answer

The evaluation includes an explicitly illustrative AI-authored answer claiming an exact calorie count. That claim has no support because standardized nutrition and dependable serving yield are absent. The chatbot responds that it cannot supply those exact values from this dataset.

Why: this directly shows the failure mode the assignment warns about: fluent numerical specificity without source support. It is not presented as an actual external-model API experiment, a comparative model ranking, or an accuracy estimate.

Output: a transparent before/after grounding example. A future empirical LLM baseline should log provider/model/version, exact prompt, raw response, settings, date, and an evidence-based judgment. No

such experiment is claimed here.

Step 25: Prepare reproducible delivery and submission materials

The project includes a short README, source package, known-answer tests, real-data evaluation, detailed roadmap, requirement mapping, AI disclosure, dataset attribution/license, and publication instructions. The offline delivery contains the built database so the user can run immediately; the public-source layout ignores generated data and caches.

Why: an interview submission must be runnable by someone who has never seen this chat. The roadmap explains the work, but the README should remain short enough to follow. Disclosure is part of the brief and should be visible.

Output: a source project and packaged deliverable. Public-repository state is recorded in `SUBMISSION_STATUS.md`. After the user selected `quackapuss/recipebox-grounded-chatbot` and authorized Git Credential Manager's official GitHub device sign-in, the source was pushed with local Git. The pre-existing MIT license and initial commit were preserved by merging histories. Anonymous cloning and GitHub Actions verified the published source. The generated database and downloaded archive remain outside Git; setup reproduces them from the official source.

3. Every framework, algorithm, and tool used

Runtime software inventory

Item	Category	Where used	Why it was used
Python 3.11+	Programming language/runtime	Entire package	Portable standard-library implementation; easy interview walkthrough
SQLite	Embedded relational database	`store.py`	Complete local record store, parameterized filtering and exact computation
SQLite FTS5	Full-text search extension	`search` table	Local title/ingredient lexical index
Porter + Unicode61	FTS tokenizers	FTS5 schema	Word normalization for discovery without changing ingredient identity
BM25	Lexical relevance ranking	`Store.search`	Rank matching records; title weight 5, ingredients weight 1
Python `sqlite3`	Database adapter	`store.py`	Execute bound queries and open runtime database read-only
Python `dataclasses`	State structure	`chat.State`	Explicit per-session fields and defaults
Python `re`	Pattern library	Router and normalization	Bounded grammar, token boundaries, ingredient/text segmentation
Python `unicodedata`	Unicode normalization	`normalize.clean`	Consistent Unicode forms before matching
Python `fractions.Fraction`	Rational parsing	Quantity parser	Parse common fractions without hand-written division logic
Python `statistics`	Statistical computation	Analytics	Mean and median from the full selected population
Python `collections.defaultdict`	Grouping utility	Quantity checks	Aggregate measurements by normalized product
Python `functools.lru_cache`	Bounded memoization	Term patterns	Reuse compiled family patterns without repeated normalization
Python `json`	Serialization	Input, output, originals, plans	Preserve source values and create machine-readable evidence
Python `hashlib`	Checksums	Ingestion and setup	Source SHA-256 manifest; observe exact-content duplicates
Python `urllib.request`	HTTP client	Setup download	Download official archive without a requests dependency
Python `zipfile`	Archive handling	Setup	Read only expected archive members
Python `pathlib`	Filesystem paths	CLI, build, evaluation	Portable path handling
Python `argparse`	CLI parser	`__main__.py`	Setup/chat/ask/evaluate commands and flags

Item	Category	Where used	Why it was used
Python `copy`	State copy	Chat execution	Roll back user-input errors atomically
Python `unittest`	Testing framework	`tests/test_system.py`	Offline, reproducible assertions with no added dependency
Python `tempfile`	Test isolation	Unit fixtures	Temporary miniature databases with automatic cleanup
`platform`, `time`, `datetime`	Runtime metadata/timing	Evaluation	Record executed environment and scenario timings
GitHub Actions	CI configuration	`.github/workflows/tests.yml`	Define repeatable fixture tests across Python versions
setuptools / PEP 517 metadata	Optional packaging	`pyproject.toml`	Standard package metadata; direct module execution needs no installation

No web application framework was used. The CLI satisfies the brief's interface requirement. No LangChain, LlamaIndex, FAISS, Chroma, embedding model, neural model, Gemini SDK, OpenAI SDK, or runtime LLM API was used. These names are not hidden dependencies.

Architectural and reasoning methods implemented in code

Method	Concrete application	Important boundary
Retrieval-grounded response design	Retrieve records before returning source facts	This is a deterministic retrieval-and-rendering system, not an LLM RAG pipeline
Relational population filtering	Single shared predicate for search/matching/statistics	Displayed top-k never defines an aggregate population
Explicit state machine	Action router updates a dataclass session	Memory is in-process and structured, not persistent across restarts
Constraint-first retrieval	Exclusions and maximum steps before ranking	Text screening is not allergy certification
Conservative entity normalization	Bounded aliases and product-form retention	No universal food ontology or fuzzy pantry equivalence
Set algebra	Matched/missing/unused pantry products	Quantities are a separate layer
Lexicographic multi-criteria ranking	Visible preference-specific tuple	No trained score or hidden weights
Jaccard similarity	Shared versus union product sets for local variants	Similarity is not a confidence probability
Evidence-gated adaptation	Related title, shared ingredients/method, plausible added role product	Observational feasibility, not causal swap proof
Provenance and reproducibility	Stable IDs, raw values, checksums, recorded calculations	Source records retained rather than deduplicated dishes
Abstention	Reject unavailable facts and insufficient evidence	Lower coverage is accepted to avoid unsupported claims

Development and document-production tools

Codex assisted with reading the brief, architecture, code, testing, debugging, and documentation. Command-line Python and PowerShell were used to run the code and inspect artifacts. `curl` downloaded the brief and official archive. The official Python and SQLite documentation was consulted. The connected GitHub tool checked access and monitored Actions. The user created the public repository, and local Git published the source after Git Credential Manager device authorization. Git merged the existing license commit without replacing history, and an unauthenticated clone verified public access. Git and Git Credential Manager are development/publication tools, not chatbot runtime dependencies.

For document production, `pypdf` extracted source PDF text, Poppler rendered PDF pages, and ReportLab generated the accompanying roadmap PDF. These are authoring tools, not chatbot runtime dependencies. The PDF was visually checked after rendering. A Markdown roadmap remains included so that explanations are searchable and editable.

The plugin-management skill was used to check Drive access options when the web reader failed; the direct download then succeeded, so no additional Drive plugin installation was needed. No messages were sent to a recruiter, no assignment form was submitted, and no model API charges were incurred.

4. Database schema walkthrough

`recipe` is the authoritative record table. Its primary key is the namespaced ID. It stores the original title, original ingredient JSON list, original instruction string, full original JSON value, source identity, usable ingredient-line count, derived step count, and searchable-validity flag.

`ingredient` stores one row per usable original ingredient line: recipe ID, original position, raw text, and normalized exact product name. The recipe index makes per-record evidence retrieval efficient; the name index supports product-based operations.

`restriction` records precomputed dairy, egg, and peanut text-screen hits. It exists to avoid rerunning many regular expressions across every recipe on every turn. A restriction-term index and composite primary key support fast existence tests.

`search` is an FTS5 table with an unindexed ID plus title and normalized ingredient text. It is a discovery index, not the raw evidence source. A retrieved ID is resolved back to `recipe` before response rendering.

`metadata` stores the manifest as JSON. The application can report which exact source files and normalization version generated its database. Runtime connections open in SQLite read-only mode; setup is the intentional writer.

5. Request walkthroughs

A search followed by a restriction

`Find a penne pasta recipe` becomes a search plan with query `penne pasta`. The store applies searchable-record status, active exclusions, and any step limit, then evaluates the lexical query. It computes the eligible count independently of the five-result display and ranks eligible IDs with BM25. The response copies source titles and ingredient evidence.

`I cannot have dairy` becomes a refinement with `dairy` added to exclusions. The query remains `penne pasta`. The same population is recomputed under the new restriction. Dairy-containing original recipes disappear. The response does not claim medical safety; it reports ingredient-text screening.

A complete-population statistical request

`How many recipes contain both garlic and onion?` sets a fresh ingredient-filter population with garlic AND onion, clears a previous unrelated dish query, and preserves active restrictions. Every searchable recipe satisfying those filters contributes once. The percentage denominator is all searchable source records, so the response states that denominator explicitly even when the numerator has additional restrictions.

`What percentage of the dataset is that?` retains the garlic/onion population because the follow-up reference is recognized. The calculation runs again from state. It never asks an LLM to remember or guess the count.

A pantry request

`I have eggs, onions, tomatoes, and rice ... no peanuts ... at most six steps` sets the pantry, peanut restriction and maximum steps. Names are canonicalized for exact identity, every eligible recipe with at least one match is examined, and the chosen preference determines the sorted key. Each result lists the pantry intersection, the required set difference, and the unused pantry set.

`Exclude eggs too` adds the egg restriction without replacing the pantry or six-step limit. The engine reuses the pantry mode and recomputes eligibility. A recipe whose pantry coverage was excellent but contains egg is removed before ranking.

An evidence-backed adaptation

`/show BASE_ID` selects a real record. `Remove eggs from this recipe` adds egg to the target restrictions, identifies the egg role hypothesis, and scans eligible local same-dish variants. A passing donor is cited and its whole ingredient list and original method become the revised variant. Differences from the base are displayed. Effects on crumb or rise are predictions; the system has not baked the recipe.

If no donor passes, the response states that the local evidence does not support a reliable adaptation. A meringue-like structural egg case is especially useful for demonstrating why a blanket applesauce replacement would be unjustified.

6. How to reproduce every important result

1. Install Python 3.11+ with SQLite FTS5. No virtual environment is required because the runtime has no third-party Python dependencies.

2. Open a terminal in the directory containing the README and `recipebox` folder.

3. Run `python -m unittest discover -s tests -v`. This uses synthetic data, not the downloaded corpus.

4. Run `python -m recipebox setup`. This downloads and imports all three official files. If using the delivered offline database, this step can be skipped.

5. Inspect `data/manifest.json` after a fresh setup, or `docs/data_manifest.json` from the delivered build. Compare source hashes if you need the same snapshot.

6. Run `python -m recipebox chat`. Enter `/help`, the brief's examples, and `/scope` to inspect state.

7. Run `python -m recipebox evaluate`. This rewrites the real-data results and actual conversation transcript. A failing check causes a nonzero process exit.

8. Inspect a cited ID using `/show ID`. Compare the original ingredient list and instructions against the adapted donor or search evidence.

9. Review `REQUIREMENTS.md`, `EVALUATION.md`, and `AI_DISCLOSURE.md` before submission.

10. Publish only the source layout, verify public repository access, and follow the recorded submission status. Downloaded data and the generated database are excluded from Git by `.gitignore`.

7. Limits, tradeoffs, and extension roadmap

The highest-priority extension is broader language understanding while preserving deterministic execution. An optional LLM could convert a request to the existing validated plan schema. It should never produce SQL, fabricate recipe IDs, calculate final statistics, override exclusions, or invent source facts. Add a benchmark of paraphrases and malformed plans before adopting it.

The next extension is a versioned ingredient ontology with broader product aliases, brand/product composition, nested alternatives, and preparation forms. Gold annotations are needed to evaluate precision and recall. A hand-written alias that boosts one example but merges distinct products can degrade the system elsewhere.

Substitution could expand to evidence extraction at the instruction span and quantity level. A defensible isolated-edit system would need controlled ratio knowledge or carefully qualified external sources, structured ingredient roles, simultaneous restriction checking, and method rewriting validation. Existing variant co-occurrence alone does not solve that problem.

For larger data, streaming JSON ingestion, more precomputed token-family tables, and database profiling would reduce setup cost and broad-filter latency. Persisted session storage would support resuming chats across restarts. A web interface could be added as a separate presentation layer while keeping the same state and execution functions.

Evaluation could add blinded relevance judgments, ingredient normalization annotations, adversarial allergy phrases, unsupported user intents, and actual external-LLM baselines with model/version logs. The current small suite establishes selected behaviors and catches regressions; it is not a population-wide accuracy estimate or medical safety evaluation.

8. Interview walkthrough and exercises

Start with the README commands and one multi-turn search. Explain that the router outputs a plan, state carries restrictions, the shared predicate defines eligible records, and rendering copies evidence. Then show a statistic and prove that its population exceeds five records. Finally inspect an adaptation's base/donor IDs and explain why the evidence is indirect.

Practice these code changes: add a safe ingredient alias and a test showing a product that must remain distinct; add a minimum or maximum ingredient-count filter to state, validation and SQL; implement a new ranking preference with deterministic tie-breaking; add an unsupported paraphrase to the router without weakening restrictions; and add a new statistic whose denominator and empty-population rule are explicit.

Be prepared to explain each set operation, why BM25 is not used for aggregates, what a Jaccard score means, why ingredient lines and distinct products differ, how source duplicates affect counts, why mass cannot automatically be compared with volume, and why an observed eggless recipe does not prove a one-to-one egg substitution.

Evaluation report

Executed on 7 November 2026. This report describes actual local runs, with the dataset snapshot identified by the accompanying manifest.

The uploaded source also passed GitHub Actions on Python 3.11, 3.12 and 3.13. Public cloning without authentication succeeded. The source was published at commit `47319f25aabbc6f5710a8d4282963ce239c9508f`; later publication/documentation commits do not change application behavior.

- Unit tests: **17 / 17 passed** using a synthetic known-answer dataset.

- Full-data checks: **17 / 17 passed** across eight executed conversations.

- Runtime: Python 3.11.9, SQLite 3.45.1.

- Raw source records: **125,164**; searchable records: **122,935**; invalid/incomplete records preserved but excluded from search: **2,229**.

Dataset coverage

Source	Raw	Searchable	Invalid / incomplete
ar	39,802	39,522	280
epi	25,323	25,260	63
fn	60,039	58,153	1,886

1,695 subsequent records had an exact title/ingredients/instructions content hash already seen. These records are retained. This is a diagnostic count, not a deduplication policy.

Exact analytical results

The garlic-and-onion query returned **21,399 / 122,935 = 17.406760%**. The denominator is all searchable records. Include predicates require the normalized ingredient name to contain each whole-token phrase, so forms such as garlic powder can count toward the garlic family. Source duplicates remain separate records. An independent ordered Python ingredient scan reproduced the SQL-backed count.

The tomato-family population contained **19,016** records. Mean ingredient-line count was **13.411653**; median was **12**. These use the complete filtered population, not the five displayed recipes.

Fewest-step title	Stable ID	Counted steps
Tomato Basil Smoothie	ar:/0wx00RefE0VHzlff/GBszObKmj63Pi	1
Best Ever Kabob Marinade	ar:3wzVv1X6HjgIShD5L9GKsL1ug4SvAPi	1
Mediterranean Zucchini and Chickpea Salad	ar:3yxgq0iAqrlm2lHnLoxwJ8vQTHCd/Cm	1
Momma's Marinara Sauce	ar:5s.4S6mTsQWRNXHDFGWe6ea03Oac48C	1
Black Bean and White Corn Salad	ar:9T2HkniyS.3VZB9NmFvzrfA39fMSt..	1

Ties are resolved by stable ID. Step counts use nonempty newline paragraphs, or sentence boundaries when instructions are one paragraph. A one-step source paragraph can contain several cooking actions; this is a text-segmentation ranking, not a difficulty judgment.

Executed conversations and timing

Conversation	Seconds
1. Search, persistent dairy restriction, and original recipe	0.087
2. Exact aggregate and follow-up percentage	5.573
3. Global filtered ranking and descriptive statistics	16.256
4. Pantry matching and an additional hard restriction	5.679
5. No match and empty aggregate	0.425
6. Egg removal using local banana-bread variants	1.412
7. Weak or conflicting egg-replacement evidence	1.480
8. Unsupported nutrition and quantity-aware name matching	3.504

Timings are observed local elapsed execution for each conversation's turns. They exclude setup and the independent aggregate oracle. They are not a production latency benchmark. Full original responses are in `test_conversations.md`.

Evidence-backed adaptation

The deterministic banana-bread base was `ar:BoBTO3QUO3Ci.0kayPs8.CfDU5oEXuu`. Final status: `supported_variant`. Local donor IDs: `fn:p174TykunvO./PLjDpl03RBTaXiOEdu`.

The donor is a complete observed eggless banana-bread variant with soy-milk yogurt. The output copies its ingredient quantities and complete preparation. The flour-family similarity mapping is used for related-variant retrieval only; exact pantry identities remain distinct. This evidence is indirect and

confounded by other recipe changes. It does not prove that yogurt alone can replace eggs at a particular ratio in the base.

The weak-evidence meringue case refused to manufacture an egg replacement. This is intended behavior because a generic binding/moisture candidate does not establish the structural function needed by that selected dish.

What initially failed and why

The first full-data run passed 15 of 16 checks. The banana-bread adaptation missed a relevant Food Network variant because exact flour identity reduced relatedness below the Jaccard threshold. The fix introduced a narrowly scoped bleached/unbleached all-purpose-flour family projection for adaptation similarity, preserving strict pantry identity. The same base then retrieved the cited variant; the evidence threshold was unchanged.

The first tomato-statistics conversation took about 60 seconds. Reusing a selected-ID population for ingredient-frequency grouping reduced the observed scenario to about 16 seconds in the next run. Broad-filter queries remain slower than lexical search; these are local measurements, not guarantees.

Final checks

Check	Result
search returns real IDs	PASS
follow-up retains dish and excludes dairy	PASS
original instructions retrievable	PASS
selected-recipe Q&A cites original ingredient lines	PASS
aggregate equals independent full-record scan	PASS
follow-up denominator and scope persist	PASS
step ranking uses tomato population	PASS
average median common use same full scope	PASS
pantry five eligible existing recipes	PASS
pantry restriction follow-up persists	PASS
pantry result keys are ordered	PASS
no match does not relax restrictions	PASS
empty average is null	PASS
banana adaptation cites compatible complete variants	PASS
weak structural-egg evidence is refused	PASS
nutrition answer refuses fabrication	PASS
quantity checks exposed	PASS

One-shot answer comparison

This is an explicitly **illustrative AI-authored one-shot-style answer**, not a sampled external-model API response and not a measured model benchmark.

User: How many calories does this recipe have?

Ungrounded illustrative answer: This recipe has exactly 350 calories per serving.

Why it fails: no standardized nutrition or dependable serving-yield fields support that number.

Actual chatbot behavior: refuses to invent exact nutrition and offers the original instructions as the available source.

What worked and what remains limited

Complete-source ingestion, stable IDs, original text retrieval, persistent restrictions, exact population statistics, deterministic pantry ranking, comparable quantity reporting, supported local-variant retrieval, and weak-evidence abstention passed the recorded checks.

The language router supports a bounded grammar. Product normalization and allergen vocabulary are heuristic. Unknown brands, compound products, optional alternatives, and source noise can cause missed equivalences or conservative exclusions. Ingredient-text absence does not prove medical safety. Quantity parsing does not cover ranges, packaging, densities or yield. Substitution is whole-variant adaptation, not a validated isolated edit. No external LLM/API baseline was measured. The small suite is not a global retrieval-accuracy or food-safety certification.

Reproduce

```
python -m unittest discover -s tests -v
python -m recipebox evaluate
```

`evaluate` exits with a failure status if any recorded check fails. Check source hashes before comparing results from a newly downloaded archive.

Assignment requirement mapping

The source brief is the user-provided GenAI Intern Assignment. The table maps its requirements to concrete implementation and validation, without assigning an unsupported score.

Requirement	Implementation	Evidence / qualification
All three JSON files	`store.build`, `__main__.setup`	Missing any source is an error; manifest stores source checksums and counts
Stable IDs and originals	`source:original_hash_key`, `raw_json`, ingredient/instruction columns	Raw invalid records retained; every result can be inspected by `/show`
Runnable conversational interface	`python -m recipebox chat`	Actual terminal run plus reproducible conversation evaluation
Dish and ingredient search	FTS5 over title and normalized ingredient text; explicit ingredient family filters	Bounded natural-language router plus exact commands; no embeddings
Context and persistent restrictions	`chat.State`	New restrictions refine current mode; explicit `/allow` and `/reset`
Specific recipe factual Q&A	`/show`, ordinal selection and full stable IDs	Full original ingredient list and instructions; no generated recipe facts
Counts and percentages	`Store.analytics`	Full filtered population; searchable-record denominator and formula displayed
Common ingredients	SQL distinct recipe frequency per normalized product	Full filtered population; duplicates within recipe count once for frequency
Average and median ingredient counts	SQL population extraction plus Python statistics	Ingredient-line rule stated; empty population returns null
Ranking by steps	SQL order over full filtered population	Explicit step rule and ID tie-break
No fabricated nutrition or timing	`unavailable` route	Standardized fields absent; show original instructions instead
Pantry names and optional quantities	`matching.match`, `quantity_check`	Exact product matching; compatible parsed quantities checked; unknowns disclosed
Hard constraints and preferences	SQL exclusions before ranking; three preference policies	No silent relaxation; recipe-text screening has stated limits
Matched, missing and extra ingredients	Set differences displayed per match	Extra required ingredients are the same set as missing pantry products; unused pantry is separate
No invented Task 2 recipes	Results are existing source IDs only	Full records attached to JSON responses
Role-aware, local-first substitution	`substitute.adapt`	Same-dish anchors, shared products and methods, role candidate, Jaccard threshold
Cite substitution support	Donor ID, title, shared products and methods	Indirect variant evidence, explicitly not causal proof
Updated ingredient list and revised instructions	Complete observed donor variant	Copies donor quantities and full method; does not claim a validated isolated swap

Requirement	Implementation	Evidence / qualification
Taste/texture/preparation tradeoffs	Role hypotheses and predicted effects	Predictions labelled; no measured outcome claimed
Weak evidence and allergies	Abstention and text screening	No medical safety certification; unresolved or unknown evidence refused
At least five conversations	Eight executed conversations	`test_conversations.md` and machine-readable evaluation results
Reproducible small evaluation set	Fixture-based unit tests and fixed full-data scenarios	`tests/test_system.py` and `recipebox/evaluate.py`
One-shot answer failure	Explicit illustrative AI-authored nutrition baseline	Shows unsupported exact calories vs grounded refusal; no external API benchmark claimed
Explain decisions	`ROADMAP.md`	Algorithms, framework inventory, formulas, limitations, module walkthrough
AI assistance disclosure	`AI_DISCLOSURE.md`	States scope of Codex assistance and candidate review responsibilities
Public GitHub repo	Published at `quackapuss/recipebox-grounded-chatbot`; `PUBLISH.md`	Source upload and anonymous clone verified; existing repository license preserved
Clear short README	Root `README.md`	Two-command setup/run; offline bundle skips setup

Deliberate scope limits

The natural-language interface is a rule-based subset, not unrestricted conversational understanding. It covers the assignment examples and documents exact commands. It does not offer a general free-form culinary explanation engine.

The substitution method is conservative whole-variant adaptation. The full donor recipe is the revised output. The implementation does not assert that swapping only one product in the base while preserving all other base quantities is validated. This tradeoff prioritizes an observed, citeable preparation over unsupported synthesis.

The one-shot comparison is demonstrative, not a measured external-model experiment. API keys were not provided and no model API was invoked. A future benchmark should record provider/model/version, prompt, raw response, date, and generation parameters before evaluating grounding.

Primary references

- Dataset and download provenance: https://eightportions.com/datasets/Recipes/

- Publisher's source repository: https://github.com/rtlee9/recipe-box

- Python SQLite API: https://docs.python.org/3/library/sqlite3.html

- SQLite FTS5, tokenizer and BM25 behavior: https://www.sqlite.org/fts5.html

- Python test framework: https://docs.python.org/3/library/unittest.html

The implementation-specific thresholds, aliases, ranking priorities, and evidence policy are design decisions in this project. The documentation does not present them as findings established by the dataset publisher or the reference manuals.

AI assistance disclosure

Codex was used to read the assignment, design the architecture, write and revise the Python code, build tests, inspect real data, debug failures, and prepare the explanatory documents. The code and documentation are AI-assisted deliverables. The included comparison with a one-shot answer is illustrative and AI-authored; no external LLM API experiment was run.

The runtime chatbot does not call an LLM. Its language router, retrieval, calculations, ranking, and evidence checks are inspectable Python and SQL. Dataset facts are computed or copied from local records.

Before submission, review the roadmap, run the tests, inspect cited records, and practice modifying the code. Do not describe this implementation as work completed without AI assistance. The interview is expected to test your understanding and ability to change it.

Submission status

Prepared on 7 November 2026.

- Local implementation: complete for the documented scope.

- All three official source files: imported and evaluated.

- Unit tests: 17/17 passed.

- Full-data evaluation: 17/17 checks passed across eight conversations.

- Detailed roadmap, framework inventory, evaluation, disclosure and README: included.

- Public GitHub repository: **published** at quackapuss/recipebox-grounded-chatbot.

- Public access: verified by cloning without authentication.

- GitHub Actions: passed on Python 3.11, 3.12 and 3.13 for the uploaded source.

- Recruiter/assignment-form submission: not sent.

The source was pushed through local Git after the user completed Git Credential Manager's official GitHub device authorization as `quackapuss`. The repository's existing MIT license and initial commit were preserved by merging the histories. The first verified source commit is `47319f25aabbc6f5710a8d4282963ce239c9508f`. No downloaded dataset or generated database is tracked in Git.

The offline bundle is immediately runnable. The public repository contains source, tests, documentation, dataset attribution, and the detailed roadmap PDF. See `docs/PUBLISH.md` for the verified publication record and clone/run commands.